



UNIVERSITÀ DI PARMA

Quick introduction to Python

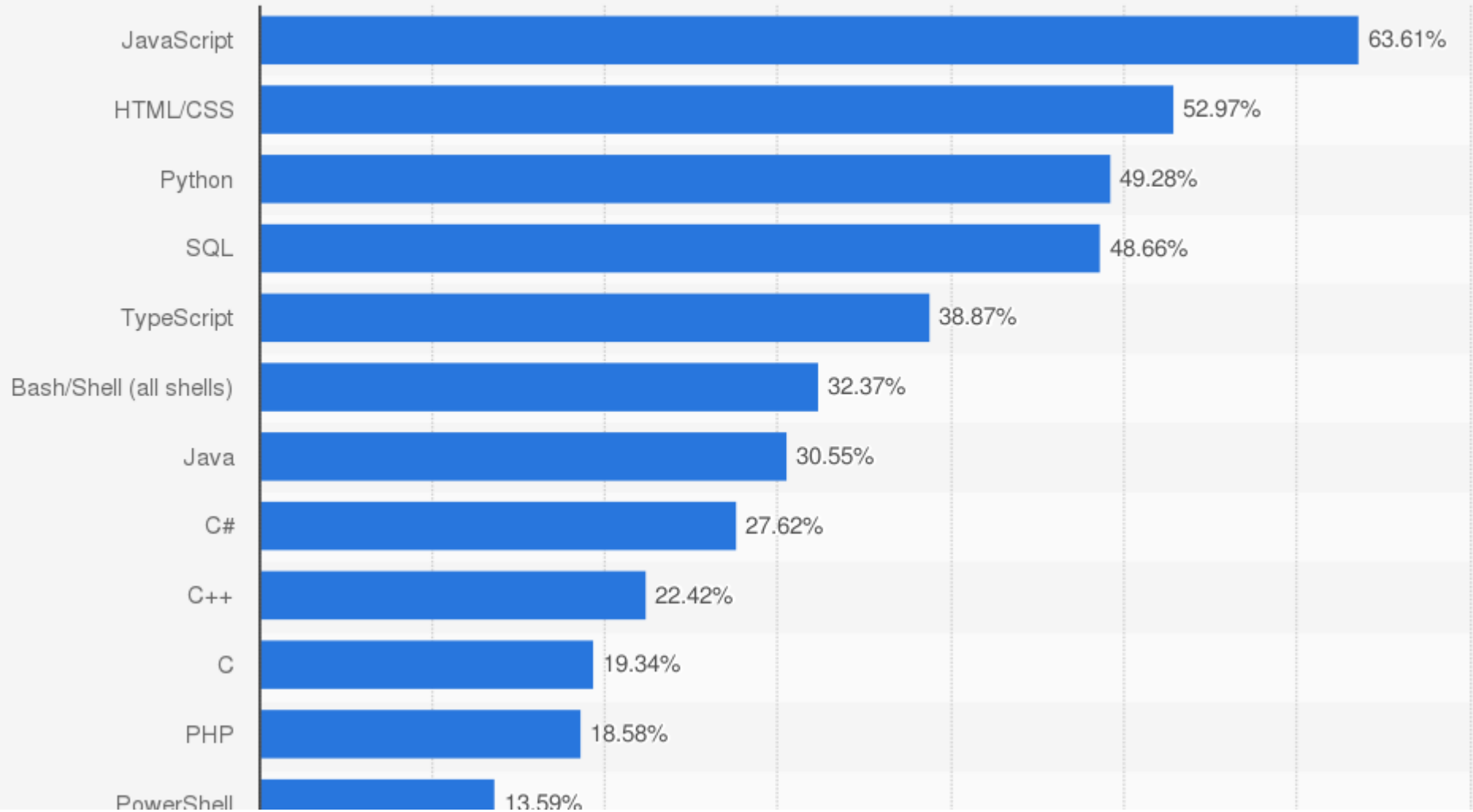


- Programming languages and their usage
- Python main features
- Basic Python elements
 - Input & Output (I/O)
 - Variables and constants
 - Expressions
 - Data structures
 - Control flow
 - Functions or subroutines

SUMMARY



Most used programming languages among developers worldwide as of 2023



- Most programming languages are
 - General Purpose
- Selecting one can be difficult
 - Specifically for learning coding
- Practically different applicability
 - C → small programs/efficiency aka embedded systems like IoT
 - C++ → large programs where efficiency is a requirement
 - Python → when business speed matters more compared to execution speed

- Started by Guido Von Rossum as a hobby
- Now widely spread
- Open Source → free
- Versatile
- Lot of additional libraries



- Widely used for scientific computing & data analysis
- One of the most important languages for
 - Data science
 - Machine & deep learning
- Hundredths of additional libraries

Top 10 Python Libraries

 Pandas Data analysis and manipulation	 NumPy Mathematical functions
 Matplotlib Data visualisations	 SeaBorn Data visualisations
 Tensorflow Machine Learning	 Keras Deep Learning
 SciPy Scientific computing	 PyTorch Machine Learning
 Scrapy Web crawling	 SQLModel Interact with SQL databases

 DATA RUNDOWN

- High level language
- Easy “quick start”
- Often used as first programming language
- Object Oriented
- Extensible
- Interpreted language

- Interpreted language
- Namely, we can directly run the code we write
- You can open a shell and directly test this
- You can install python or
 - online environments

<https://www.programiz.com/python-programming/online-compiler/>

Try it yourself!

- Open “programiz” web site and simply run the default code

<https://www.programiz.com/python-programming/online-compiler/>

- As other languages we have a **lexicon** and **syntax** rules
- Namely
 - Instructions
 - How to combine them
- Moreover
 - Do not forget **semantics**
 - What we write must also make sense

- Input & Output (I/O)
- Variables and constants
- Expressions
- Data structures
- Control flow
- Functions or subroutines

- “Notes” that can be distributed along your code
- Human readable explanations
- They improve how to understand the code
- In Python everything after a “#” until a “newline”
 - Also text inside `"""` and `"""` is ignored and then used as comment

When a programmer first creates his code, only he and God know how it works, a few months down the line and only God knows

Try it yourself!

- Add some “comments” to the default code
- Which character(s) can be used for comments?
 - %
 - //
 - #
 - /*

<https://www.programiz.com/python-programming/online-compiler/>

- “print()” instruction can be used to print on the screen
- We can also print different things separating them using a ‘,’
 - A space is then used to separate them
- A newline is added at the end of the output

```
print("Hello world!")      # print a “string”  
print(3)                  # print a “number”  
print("My age is ", 57)    # combining multiple outputs
```

- “input()” instruction can be used to read some “text” as input
- When issuing the input() an optional prompt message can be passed
- The input() stops until the enter key is pressed

```
name = input("What is your name? ") # ask user  
print("Hello", name)
```

- “input()” always read “text”, namely a sequence of symbols
 - Even when a number is passed as input
- We need to convert other data...

```
age = input("What is your age? ") # ask user
print("Oh, you look younger than", age, "years")
print("I am", age + 10, "years old")
```

```
# the last line will give you a
# “TypeError: can only concatenate str (not “int”) to str”
# error
```


- “input()” always read “text”, namely a sequence of symbols
 - Even when a number is passed as input
- We need to convert other data...

```
age = int(input("What is your age? ")) # ask user
print("Oh, you look younger than", age, "years")
print("I am", age + 10, "years old")
```

```
# now it works flawlessly
```

- Python is a Typed Programming Language
 - Actually a strong dynamically typed language
- i.e. data feature a “type”
 - Integer number, floating point number, string...

- How to write data (“simple” ones), namely constant values
 - 42 → integer, decimal
 - 042 → integer, octal
 - 0x2A → integer, hexadecimal
 - 0.15 → floating point
 - .15 → floating point
 - 1.5e2 → floating point
 - "ciao" → string

- A variable is a “container” to store data
 - i.e. it allows to interact with the memory
 - Read data
 - Write data
- In Python a variable is created when a value is assigned
 - `a = 57`
 - `b = “ciao”`
 - `pi = 3.141`

- In order to create a variable then
 - We need to choose a name
 - Only letters, digits or “_”
 - Have to start with a letter
 - Mnemonic names are to be preferred
 - No need to indicate a specific type
 - You can use “type(<variable name>)” to detect data type
 - Data type can change during variables lifetime

- Operator “=” is used to assign (store) data
 - `a = 17`
 - `b = a`
 - `x, y, z = "Orange", "Banana", "Cherry"` # multiple assignments
- The content of a variable can “vary”
 - `c = 17`
 - ...
 - `c = “ciao”`

- Combinations of **operators** and **operands**
- Mathematical
 - “+”, “-”, “*”, /, “//” (quotient), “%” (remainder), and “**” (exponential)
- Relational
 - “>”, “<”, “>=”, “<=”, “==” (equal to), “!=” (not equal to)
 - Can be chained (i.e. $0 < a < 17$)
- Logical
 - “and”, “or”, “not”

Incomplete list!

Try it yourself!

- Ask user for a radius value of a circle
- Compute both the circumference and area of the circle
 - Hints: area $\rightarrow \pi r^2$, circumference $\rightarrow 2\pi r$, $\pi \rightarrow 3.141$
- Print them

<https://www.programiz.com/python-programming/online-compiler/>

- Complex programs need to manage also “big” data or even more “complex” data than single numbers or strings
- We are going to briefly introduce Python “Collections”
 - **Lists** → ordered and changeable, duplicate members
 - **Tuples** → ordered and unchangeable, duplicate members
 - **Sets** → unordered, unchangeable, unindexed, no duplicate members
 - **Dictionaries** → ordered and changeable. no duplicate members

- Lists allow to store multiple “items” in a single variable
- Square brackets are used to define them
 - `emptylist = []`
 - `workstation = ["mouse", "computer", "keyboard"]`
 - `print(workstation)`
- Lists can be heterogeneous
 - `workstation.append(42) # appending`
 - `workstation.extend(["RAM", "monitor"]) # extending`

- In a list single elements feature an “index”
- First element $\rightarrow 0$
- We can access single elements using the index or even portions of the list

```
numbers = [1.414, 3.142, 2.718, 1.618, 1.202]
print(numbers[0])          # 1.414
print(numbers[0:2])        # [1.414, 3.142]
print(numbers[2:])         # [2.718, 1.618, 1.202]
print(len(numbers))        # 5
```

Try it yourself!

- Define the following list
- `thislist = ["apple", "banana", "cherry", "orange", "kiwi", "mango"]`
 - Print the third element
 - Change the third element as “pear”
 - Using “.sort” sort the list
 - Using “.append” add “lemon” to the list
 - Using “.remove” remove “kiwi” from the list
 - Using “.pop” remove the fourth element from the list

<https://www.programiz.com/python-programming/online-compiler/>

- Dictionaries extend the lists allowing also to set the index
- Data are stored in a **key:value** fashion

```
person = {  
    "name":      "Mario",  
    "surname":   "Rossi",  
    "IDs":       ["CI: AC2345Y", "PAT: 12349012"]  
}  
print(person)
```

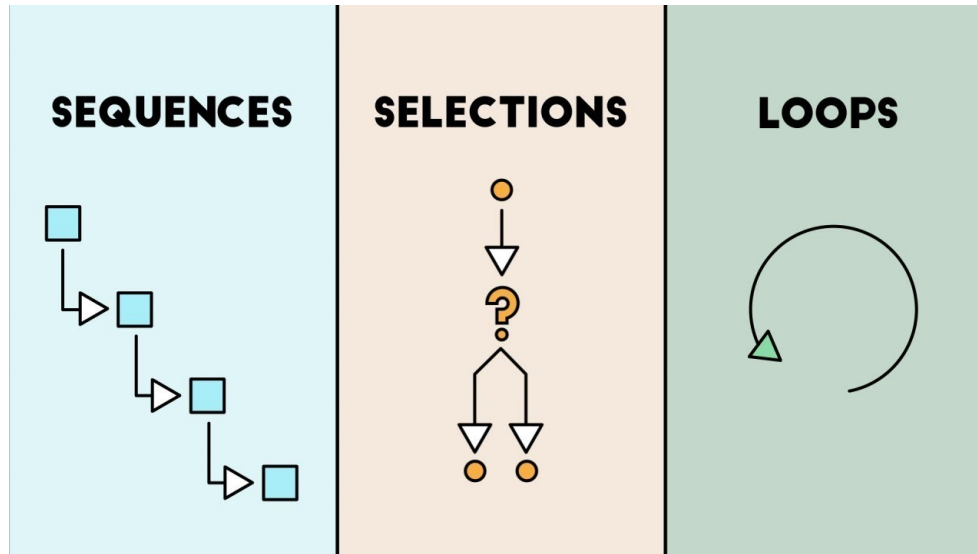
- The value of a single element can be accessed through the key
 - The key can be any **uniq** immutable object, i.e. strings, numbers, tuples
 - Values can be, as in the lists case, heterogeneous
 - `print(person["name"])`
 - `print(person.get("name"))`
 - `print(person.keys())`
 - `print(person.values())`
 - `print(person.items())`

Try it yourself!

- Use the following Python dictionary that contains phone numbers
- `tel = {"Bertozzi": 5845, "Rossi": 5722, "Lab": 5723, "Ugo": 3456}`
 - Print the phone number of Lab using `“.get()”`
 - Remove the phone number of Ugo using `“.pop()”`
 - Add a phone number
 - Remove all elements using `“.clear()”`

<https://www.programiz.com/python-programming/online-compiler/>

- Instructions are executed in the order they are met
- This is definitely limiting, we need to “control” the “flow” of execution



- Most programming languages do not care about indentation
- Humans tend to group similar things together
- Therefore indentation can be used to beautify your code
- But **for Python indentation is mandatory!**

```
# Python code
if foo: Text
    if bar:
        baz(foo, bar)
    else:
        qux()
```

- It allows to perform a different processing depending on evaluating a condition

```
temp = int(input("Give me a temperature: "))  
if (temp >= 100):  
    print("Put pasta in the water!")  
elif ( temp > 90):  
    print("A bit early but just few minutes")  
else:  
    print("Do not be so impatient!")
```

Try it yourself!

- Ask user an integer number
- Then print if the number is even or odd

<https://www.programiz.com/python-programming/online-compiler/>

- Python provides two main possibilities for loops
- **for** loop
 - Typically used for a “fixed” amount of iterations
- **while** loop
 - Typically used until a specific condition occurs

- Used for iterating over a sequence
 - And to execute something each iteration

```
for x in range(10): # 0-9
    print(x)
```

```
fruits = ["Apple", "Orange"]
for fruit in fruits:
    print(fruit)
```

- For iterating over a dictionary we can use:

```
provinces = { "PR": "Parma", "MI": "Milano", ...}  
for acronym, province in provinces.items():  
    print(acronym, province)  
else:  
    print("No more provinces!")
```

- Execute statements as long as a condition is true

```
i = 1
while i < 6:
    print(i)
    i += 1 # += increment and assign values at once
else:     # optional part
    print("Now the count reached:", i)
```

Try it yourself!

- Use a *for* and a nested *if* to print odd numbers up to 99
- Rewrite previous code using a *while* loop
 - Is the *if* still necessary?

<https://www.programiz.com/python-programming/online-compiler/>

- Building blocks that ease a program development
 - Sub-problems matching
 - Easier to debug
 - Define once, use every time you need

- The “def” keyword is used to define a function

```
def hello():  
    print("Hello World!")
```

```
# to call a function, simply use function name + ()  
hello()  
  
# you can do this as many times as you need  
hello()
```

- Data can be passed into functions as arguments
- Or returned using the “return” or “yield” instructions

Positional

```
def add(x, y):  
    return x + y
```

Keyword

```
def shout(phrase='Help me!'):  
    print(phrase)
```

Default

```
def echo(text, prefix = ''):  
    print(prefix, text)
```

- Compute Fibonacci numbers up to n

```
def fib(n):  
    results = []  
    a, b = 0, 1  
    while a < n:  
        results.append(a)  
        a, b = b, a + b  
    return results
```

- Continuous Fibonacci computation

```
def fib():  
    a, b = 0, 1  
    while True:  
        yield a  
        a, b = b, a + b  
  
for value in fib():  
    print(value)
```

Try it yourself!

- Define a function that find the maximum between two numbers and return it
- Define a function that sum all the numbers in a list

<https://www.programiz.com/python-programming/online-compiler/>

- Python has other features:
 - Classes → OOP programming
 - Imports → code isolation and re-use
 - Error handling → exceptions
 - Docstrings → support for documenting all modules
 - A lot of nice additional modules!



UNIVERSITÀ DI PARMA

Quick introduction to Python

